

STM32N6 Nucleo YOLOv2-tiny Porting Plan

Target board: NUCLEO-N657X0-Q

Current project: stm32n6-gettingstarted-objectdetection

Goal: replace or regenerate the current Tiny YOLOv2 model while keeping the existing Nucleo application structure stable.

1. 결론

현재 프로젝트를 크게 바꾸지 않고 진행하려면 가장 안정적인 조합은 **YOLOv2-tiny, 224 x 224 x 3 input, uint8 input, signed int8 output**입니다. 이 조합은 현재 Nucleo 예제의 모델, DCMIPP 입력 크기, post-processing 설정과 가장 잘 맞습니다.

항목	권장값	이유
Model family	YOLOv2-tiny	현재 예제가 이미 Tiny YOLOv2 계열 모델과 YOLOv2 후처리를 사용합니다.
Input size	224 x 224 x 3	현재 STAI_NETWORK_IN_1_WIDTH/HEIGHT가 224이고, 224 / 32 = 7이라 기존 7 x 7 grid 설정과 맞습니다.
Input tensor type	uint8	현재 생성 스크립트가 <code>--input-data-type uint8</code> 을 사용합니다.
Output tensor type	signed int8	현재 생성 스크립트가 <code>--output-data-type int8</code> 을 사용하며, postprocess는 <code>POSTPROCESS_OD_YOLO_V2_UI</code> 입니다.
Display backend	UVCL 기본	Nucleo 보드는 LCD가 없으므로 기본 출력은 USB/UVC입니다. 외부 SPI LCD를 쓰면 SPI backend를 선택합니다.

2. 현재 프로젝트 구조 요약

Nucleo 예제의 `main.c`는 카메라 입력, DCMIPP pipe, neural network inference, post-processing, UVC/SPI 표시 출력을 모두 연결합니다.

Camera sensor

```
-> DCMIPP PIPE1 -> lcd_bg_buffer -> screen1 -> UVC/SPI display output  
-> DCMIPP PIPE2 -> nn_in          -> STAI/NPU inference -> postprocess -> overlay
```

lcd_bg_buffer와 lcd_fg_buffer는 neural network 입력 버퍼가 아니라 UVC 또는 외부 SPI LCD로 보낼 화면 출력용 layer buffer입니다.

3. 현재 모델 확인

현재 Nucleo 모델은 quantized_tiny_yolo_v2_224_.tflite에서 생성된 Tiny YOLOv2 계열입니다.

```
stedgeai generate \  
--model quantized_tiny_yolo_v2_224_.tflite \  
--target stm32n6 \  
--st-neural-art default@user_neuralart_NUCLE0-N657X0-Q.json \  
--input-data-type uint8 \  
--output-data-type int8
```

현재 input tensor 정보는 다음과 같습니다.

```
#define STAI_NETWORK_IN_1_FORMAT (STAI_FORMAT_U8)  
#define STAI_NETWORK_IN_1_SIZE   (150528)  
#define STAI_NETWORK_IN_1_CHANNEL (3)  
#define STAI_NETWORK_IN_1_HEIGHT (224)  
#define STAI_NETWORK_IN_1_WIDTH  (224)
```

계산:

```
224 x 224 x 3 = 150,528 bytes
```

4. 모델 교체 작업 계획

Step 1. 모델 준비

- YOLOv2-tiny 구조를 사용합니다.
- 입력 크기는 가능하면 224 x 224 x 3으로 유지합니다.
- class 수와 anchor 값을 학습 설정에서 기록해 둡니다.

- 양자화는 input uint8, output signed int8 조합으로 맞춥니다.

Step 2. ST Edge AI 생성

새 모델 파일 이름을 예로 my_yolov2_tiny_224_int8.tflite라고 하면:

```
stedgeai generate \  
  --model my_yolov2_tiny_224_int8.tflite \  
  --target stm32n6 \  
  --st-neural-art default@user_neuralart_NUCLE0-N657X0-Q.json \  
  --input-data-type uint8 \  
  --output-data-type int8
```

생성 후 현재 Nucleo 모델 폴더로 복사해야 하는 파일:

st_ai_output/network.c	-> Model/NUCLE0-N657X0-Q/network.c
st_ai_output/network_ecblobs.h	-> Model/NUCLE0-N657X0-Q/network_ecblobs.h
st_ai_output/stai_network.c	-> Model/NUCLE0-N657X0-Q/stai_network.c
st_ai_output/stai_network.h	-> Model/NUCLE0-N657X0-Q/stai_network.h
st_ai_output/network_atonbuf.xSPI2.raw	-> Model/NUCLE0-N657X0-Q/network_data.xSPI2.bin

그리고 external flash용 hex를 다시 만듭니다.

```
arm-none-eabi-objcopy \  
  -I binary Model/NUCLE0-N657X0-Q/network_data.xSPI2.bin \  
  --change-addresses 0x70380000 \  
  -O ihex Model/NUCLE0-N657X0-Q/network_data.hex
```

Step 3. app_config.h 수정

YOLOv2 signed int8 output을 유지한다면 postprocess는 그대로 유지합니다.

```
#define POSTPROCESS_TYPE POSTPROCESS_OD_YOLO_V2_UI
```

새 모델에 맞게 아래 값을 반드시 확인합니다.

```
#define NB_CLASSES  
#define CLASSES_TABLE  
#define AI_OD_YOLOV2_PP_NB_CLASSES  
#define AI_OD_YOLOV2_PP_NB_ANCHORS  
#define AI_OD_YOLOV2_PP_GRID_WIDTH
```

```
#define AI_OD_YOLOV2_PP_GRID_HEIGHT
#define AI_OD_YOLOV2_PP_ANCHORS
#define AI_OD_YOLOV2_PP_CONF_THRESHOLD
#define AI_OD_YOLOV2_PP_IOU_THRESHOLD
#define AI_OD_YOLOV2_PP_MAX_BOXES_LIMIT
```

224 x 224 YOLOv2-tiny이면 보통 output grid는 7 x 7입니다. 416 x 416을 쓰면 보통 13 x 13이므로

AI_OD_YOLOV2_PP_GRID_WIDTH와 AI_OD_YOLOV2_PP_GRID_HEIGHT를 바꿔야 합니다.

Step 4. 빌드 및 플래시

모델 weights/biases인 `network_data.hex`를 먼저 external flash에 씁니다.

```
STM32_Programmer_CLI \
  -c port=SWD mode=HOTPLUG \
  -el <STM32CubeProgrammer_N6>/bin/ExternalLoader/MX25UM51245G_STM32N6570-NUCLE0.stldr \
  -hardRst \
  -w Model/NUCLE0-N657X0-Q/network_data.hex
```

그 다음 application binary를 다시 빌드하고 플래시합니다.

5. UVC/LCD 관련 정리

NUCLE0-N657X0-Q에는 onboard LCD가 없습니다. 현재 프로젝트의 LCD 이름은 ST의 generic display API 이름입니다. 실제 출력 backend는 build 설정으로 정해집니다.

```
# Default Nucleo output
SCR_LIB_SCREEN_ITF := UVCL

# External SPI display shield
SCR_LIB_SCREEN_ITF := SPI
```

따라서 Nucleo 기본 구성에서는 inference 결과가 bounding box/text overlay와 함께 USB/UVC video stream으로 PC에 전달됩니다.

6. 검증 체크리스트

검증 항목	확인 방법	통과 기준
ST Edge AI generation	<code>stedgeai generate</code> 실행 로그 확인	unsupported operator 없이 완료
Input shape	<code>stai_network.h</code> 확인	224 x 224 x 3, U8
Output quantization	<code>stai_network.h</code> 또는 생성 로그 확인	postprocess 설정과 일치
Postprocess parameter	<code>app_config.h</code> 확인	class 수, anchors, grid size가 학습 모델과 일치
External flash data	<code>network_data.hex</code> flash	flash 성공 및 부팅 시 network load 실패 없음
Runtime result	UART log와 UVC camera app 확인	inference time 출력, object box 표시

7. 주요 리스크

가장 흔한 실패 원인은 모델 output tensor layout과 postprocess 설정 불일치입니다. 같은 YOLOv2-tiny 라도 training/export 방식에 따라 anchors, class 수, output order가 다를 수 있습니다.

- ST Edge AI가 모델의 모든 operator를 지원하지 않을 수 있습니다.
- 양자화 scale/zero-point가 postprocess 기대값과 맞지 않으면 detection 결과가 깨질 수 있습니다.
- 416 x 416 같은 큰 입력은 메모리와 FPS 부담이 커집니다.
- class table과 AI_OD_YOLOV2_PP_NB_CLASSES가 불일치하면 box label 또는 class index 문제가 생길 수 있습니다.

8. 추천 진행 순서

1. 현재 기본 모델로 board, camera, UVC 출력이 정상인지 먼저 확인합니다.
2. 새 YOLOv2-tiny 224 x 224 모델을 input uint8, output signed int8로 준비합니다.

3. ST Edge AI generation이 되는지 먼저 확인합니다.
4. `stai_network.h`의 input/output shape를 확인합니다.
5. `app_config.h`의 class, anchor, grid, threshold를 모델에 맞춥니다.
6. `network_data.hex`와 application binary를 모두 다시 flash합니다.
7. UART log와 UVC 화면에서 inference 결과를 확인합니다.